Attention-based Value Classification Reinforcement Learning for Collision-free Robot Navigation

Chao Sun, Xing Wu, *Member, IEEE*, Yuanda Wang, Changyin Sun, *Senior Member, IEEE*

***Abstract*—Collision avoidance is a crucial technique to achieve safe and efficient robotic vehicle navigation in unknown environments. However, moving obstacles with unpredictability in dynamic scenarios, usually increase the difficulty and complexity in collision avoidance of robotic vehicles. To enhance the stability of collision avoidance and boost its adaptability to uncertain dynamic scenes, a new attention-based value classification actor-critic (AVCAC) architecture is proposed. It is an end-to-end robot navigation model that utilizes imperfect local observation to directly plan accurate collision-free motion commands. First, we design a value-classified rollout replaybuffer to categorize the experiences into different pools. It can prevent any overfitting or bias that may result from repeatedly sampling experiences of a certain type during policy learning. Then, we improve the conventional actor-critic network with a multi-head local attention module to extract the local observations at entity-level. This way, the collision avoidance system can focus on key environmental features to operate more efficiently and respond more swiftly to dynamic changes in the environment. Moreover, a lookahead multi-step prediction (LMP) reward setting is devised in the AVCAC-based reinforcement learning (RL) framework to facilitate more informed and forward-looking decision-making. Finally, the policy entropy (PE) and policy delay (PD) are extended to AVCAC model to enhance policy exploration and make policy more robust. Extensive experimental results reveal that our method can generate time-efficient and collision-free guide paths to dodge collisions under complex dynamic environments.**

***Index Terms*—Robotic vehicle, collision-free navigation, reinforcement learning, value classification, local attention.**

I. Introduction

AUTOMATED guided vehicle (AGV) is one typical industrial robotic vehicle utilized for repeated transportation tasks in factories for more than half of a century [1], [2]. With the wide application of AGV in disaster rescue, restaurant delivery, and household service, the collision avoidance methods have become increasingly important during AGV navigation. It plays a vital role in enabling AGVs to operate autonomously and safely in the environments. While there has been significant progress over the past decade [3], [4], fully and safely autonomous navigation is still challenging, especially in dynamic and uncertain workspaces cohabited by other moving objects (such as pedestrians, vehicles, or other robots). The challenge usually arises due to the lack of knowledge of the navigation system about the intentions and strategies of other dynamic objects. Therefore, the system endows AGV with the capabilities of obstacle interaction and collision risk assessment, which are of great significance for AGV to plan a collision-free path in complex dynamic scenes.

Classical collision avoidance methods can be divided into global and local approaches according to the task hierarchy. The former is based on the known obstacles map to generate continuous paths to avoid collisions [5], [6]. This process involves two crucial points. An environment map with obstacles is first built offline/online via simultaneous localization and mapping (SLAM) technique, followed by planning a collision-free trajectory with respect to the map for safe navigation. Yet, in a scene full of moving obstacles, SLAM may fail frequently because of noises or occlusions, resulting in the inability to generate reliable maps [7]. Besides, the SLAM is difficult to be implemented on the robot platforms with limited computation resource, due to the expensive map construction.

Given the above limitations, the local collision avoidance navigation techniques have been proposed recently. It is one kind of mapless navigation solution, in which AGVs typically utilize the real-time imperfect local sensing information rather than the prior knowledge of the surrounding environment to generate temporary avoidance paths for safe and efficient navigation [8], [9]. In this context, this group of techniques is more applicable to practical robotic application for collision-free navigation. Hence, the local collision avoidance method is focused on this paper.

According to the technical route, conventional mapless navigation methods can be divided into two strategies. One is the geometric rule strategy. It simplifies the obstacles as convex bodies, followed by computing the lower and upper boundaries of collision-free navigation path. The typical methods include artificial potential field (APF) [10], [11] and reachability analysis (RA) [12], [13]. APF method and its variants rely primarily on the repulsive force engendered when approaching obstacles to drive the AGV to dodge collisions. While APF-based solutions are efficient and easy to implement, many of them neglect future behavior of dynamic obstacles. It may cause troubles in AGV collision avoidance when the dynamic changes of moving obstacles are large. To alleviate this issue, Malone *et al*. [14] improve the APF with a Markov-based future event prediction module to assess collision threat, but

This work was supported in part by the National Key Research and Development Program of China under Grant 2018AAA0101400; in part by the National Natural Science Foundation of China under Grant 61973154 and Grant U1713209; in part by the QingLan Project Funding for Universities in Jiangsu Province, China; and in part by the National Defense Basic Scientific Research Program of China under Grant JCKY2022209B001. *(Corresponding author: Changyin Sun and Xing Wu.)*

Chao Sun is with the College of Electronics and Information Engineering, Tongji University, Shanghai 201804, China (e-mail: 2010466@tongji.edu.cn).

Xing Wu is with the College of Mechanical and Electrical Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China (e-mail: wustar5353@nuaa.edu.cn).

Yuanda Wang and Changyin Sun are with the School of Automation, Southeast University, Nanjing 210096, China (e-mail: wangyd@seu.edu.cn; cysun@seu.edu.cn).

leading to a high computational cost.

The RA is a reachable set-based obstacle avoidance solution. The reachable set is calculated by polar technique and Hamilton-Jacobi (HJ) formulation. It represents a kind of target set that the robot in a certain initial state can safely reach through control in the presence of any disturbance. Chen *et al*. [15] compute the reachable set using the polar algorithm to achieve collision avoidance, in which the obstacle avoidance set is defined as the complement of controlled invariant. In [16], a robust collision-free navigation method is investigated by constructing the forward and backward reachable sets using HJ formulation. While these solutions adopt the Markov process approximation to decrease the runtime of probability computation for reachable sets, they struggle to cope with highly-dynamic scenes existing many moving obstacles.

The second strategy is the path searching techniques that search out a reasonable path via optimizing. The works [17], [18] use the nonlinear model prediction control (NMPC) methods to realize obstacle avoidance. This way can combine various constraints to gain optimal control and real-time operation in the process of collision avoidance. Yet, as the number of obstacles increases, the vector dimension of constraint increases rapidly, which can decrease the performance of NMPC dramatically. In [19], [20], researchers design the dynamic window approach (DWA) mechanism to gain the samples from the velocity space of the AGV, followed by predicting its possible motion trajectory via an objective function. Missura *et al*. [21] present an arc-based DWA to model the moving obstacles as polygonal objects with velocity, and predict future collision threats, which achieves agile and feasible obstacle avoidance. Nevertheless, these studies easily plunge into local minima, since DWA solely considers the AGV's goal heading and lacks free-space connection information.

With the progress of artificial intelligence (AI), many RL-based mapless navigation methods have gained attention, which map raw local observation to collision-free steering commands directly [22]–[24]. This kind of solutions is one of the machine learning algorithms, which can learn and optimize the navigation policy in a "trial and error" fashion, through interacting with the environments. Unlike conventional methods, it has a strong adaptability to model-free feature and complex systems, which provides a new method to the collision-free navigation problem. In [25], a transformer-based dual-channel self-attention term is proposed to endow the AGV with the ability to assess collision threats, thereby making collision avoidance decisions. He *et al*. [26] combine the asynchronous multithreading mechanism with RL to achieve safe and autonomous motion planning. The work [27] introduces an information-theoretic regularization term into RL objective to promote more informed navigation decisions. Wu *et al*. [28] use grid-map images as state inputs to develop dynamic collision situations. However, the aforementioned studies only perform well in relatively simple and structured test environments. Hence, these RL-based navigation models are hard to generalize to complex and highly dynamic scenes.

To tackle this challenge, a discrete soft actor critic method that combines data replay and attention encoding is implemented to enhance the performance of RL [29]. In [30], [31],

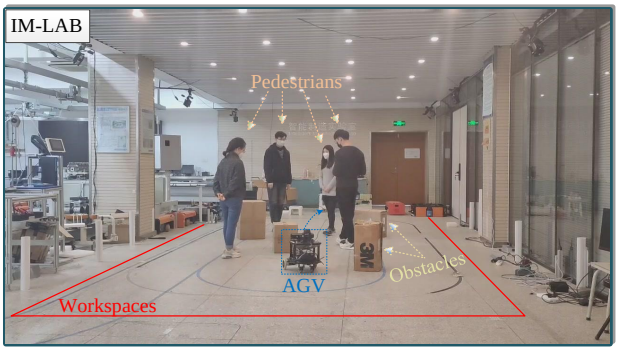


Fig. 1. AGV navigation in real complex dynamic environments.

researchers utilize the global information to guides the update of the decentralized actor network in light of assuming that each robot fully understands the environment. This setting is suitable for processing small scenario tasks. In real application, the field of view (FOV) of each AGV is limited by the sensing range of dominant sensor. Therefore, some researchers select raw sensor data (e.g., Lidar [32] or Camera [33]) as inputs to develop end-to-end RL patterns for realizing AGV collision avoidance in real scenes. The sensor-level techniques can deal with the variable observation dimensions well, but introduce interpretability and large-size input problems. Furthermore, Long *et al*. [34] use multi-scene multi-stage training paradigm to estimate the states of moving obstacle for learning an optimal policy. In [35], a Bayesian inference method is devised to predict surrounding dynamic collision in advance and calculate collision-free actions using optimal reciprocal collision avoidance framework. In [36], scholar utilizes supervised learning algorithms to train a system that learns from labeled examples of collision-avoidance scenarios. The actual performance of this way depends heavily on the quality of the labeled data or presentation. In summary, existing collision-free decision methods still have many limitations, when operating in the imperfect sensing, large-scale search, and dynamic uncertain workspaces.

To address the problems mentioned above, a novel attention-based value classification actor-critic (AVCAC) collision avoidance method is proposed in this paper. It is a model-free end-to-end robotic vehicle navigation method that can enable the AGV to make more informed decisions for navigating safely and efficiently in complex dynamic scenes (see Fig. 1). Our major contributions are as follows:

(1) A novel RL method that extends policy delay and policy entropy to the actor-critic paradigm is built to learn optimal collision avoidance policy. This way can directly leverage the local observation information to compute collision-free motion commands without perfect perception condition, thus ensuring safe and efficient navigation of AGV in dynamic uncertain scenarios.

(2) A value-classified rollout replaybuffer is devised to reduce the variance and enhance the sample utilization. Also, a state encoder called local attention module is developed to extract and encode the local information at entity-level. It can provide a more lightweight representation and capture

critical environmental features relevant to collision avoidance tasks. Through the tight cooperation between rollout replaybuffer and state encoder, the navigation system can improve its perception capability and learning efficiency, thus making safe and effective collision avoidance decisions rapidity.

(3) We innovatively introduce a lookahead multi-step prediction (LMP) reward function in the RL system during training phase to predict future events of collision threat. It prevents the navigation system from generating short-sighted, intrusive, and unnatural motion policies.

The rest of this paper is organized as follows. Section II formulates the collision avoidance problem and introduces the AGV control model. Section III shows a detailed discussion of our AVCAC model. The experiment results are presented in Section IV. Finally, conclusions and recommendations for future research are given in Section V.

II. Problem Formulation

From the perspective of RL, the collision avoidance navigation task can be modeled as a partially observable Markov decision problem (POMDP) [37]. Formally, the POMDP is described as a 6-tuple $(\mathcal{A}, \mathcal{S}, \mathcal{R}, \mathcal{T}, \mho, \mathcal{O})$, where $\mathcal{A}$ is action space ($\boldsymbol{a} \in \mathcal{A}$), $\mathcal{S}$ is global state of environment ($\boldsymbol{s} \in \mathcal{S}$), $\mathcal{R}$ is the reward function ($r \in \mathcal{R}$), $\mathcal{T}$ is the transition probability between states $\mathcal{T}(\boldsymbol{s}' | \boldsymbol{s}, \boldsymbol{a})$, $\mho$ is observation space ($\boldsymbol{x} \in \mho$), and $\mathcal{O}$ is the observation distribution given the state ($\boldsymbol{x} \sim \mathcal{O}(\boldsymbol{s})$).

In the process of collision avoidance task, the AGV leverages the navigation policy learned from its own limited local observations $\boldsymbol{x}_t$ to execute the action $\boldsymbol{a}_t$ at each timestep t. Then, the environment reward feedback is gained, and while the navigation system reaches the next state. The goal of this process is to drive the AGV to approach target autonomously and safely, which corresponds to learning an optimal collision-free policy that maximizes the expected return E, that is:

$$E = \sum_{k=1}^{\infty} \lambda^{k-1} r_{t+k} \tag{1}$$

where $\lambda \in [0, 1]$ denotes the discount factor, which weighs the long-term gain and short-term reward.

It is noteworthy that the AGV tends to decelerate or accelerate abruptly during collision avoidance in real applications, since there is a sudden change between the steering command $\boldsymbol{a}_t$ output by the navigation policy and the current velocity $\boldsymbol{v}_t$. This can cause the AGV to generate an unsmooth travel trajectory. To this end, a commonly-used proportion integration differentiation (PID) control method is embedded into our navigation system as the underlying action control strategy. It enables the AGV to take advantage of the stability of classical control algorithm for final executable, smooth, and continuous trajectory traveling. It can be formulated as:

$$\boldsymbol{a}_t^{re} = \boldsymbol{v}_t + \boldsymbol{u}_t; \boldsymbol{u}_t = K_p \boldsymbol{e}_t + K_i \int_{t_s=0}^{t} \boldsymbol{e}_t d_t + K_d \frac{d\boldsymbol{e}_t}{dt}; \boldsymbol{e}_t = \boldsymbol{a}_t - \boldsymbol{v}_t \tag{2}$$

where $\boldsymbol{a}_t^{re}$ is the real velocity after being processed by PID controller, t_s is the starting time, i.e., $t_s = 0$. K_p, K_i, and K_d are the coefficients of proportion, integration and differentiation, respectively. Thereafter, AGV's self-states (position and heading angle) at the next timestep are updated by:

$$\begin{bmatrix} p_{t+1}^x \\ p_{t+1}^y \\ \beta_{t+1} \end{bmatrix} = \begin{bmatrix} p_t^x \\ p_t^y \\ \beta_t \end{bmatrix} + \begin{bmatrix} \cos\beta_t \cdot v_t^l \\ \sin\beta_t \cdot v_t^l \\ v_t^w \end{bmatrix} \cdot \Delta t \tag{3}$$

where β_t is the AGV's heading angle. p_t^x and p_t^y are the horizontal and vertical components of the AGV's position $\boldsymbol{p}_t$, respectively, v_t^l and v_t^w are the translational and rotational velocities respectively, i.e., $\boldsymbol{a}_t = \left[v_t^l, v_t^w\right]$.

III. Methodology

The overview architecture of our AVCAC method is shown in Fig. 2. First, the AGV with imperfect perception navigates to its goal in a complex environment full of uncertain moving obstacles (e.g., pedestrians). In the process of navigation, AGV interacts with the environments to summarize the experience information for generating rollout data and transferring them into the valued-classified rollout replaybuffer (VCRR). Through VCRR mechanism, these rollout data is partitioned and stored into different pools with fixed timestep sequence. This trick is much more practical and can increase the utilization of sample effectively. Then, we design a local attention-based state encoder to detect and encode the critical environmental feature at entity-level. It can measure the relative importance of surrounding (static and dynamic) obstacles to assist the AGV make better navigation decisions. Furthermore, the local encodings are sent into the value (critic) networks, which obtain the state-action Q value by introducing PE $\log \pi$. Meanwhile, the policy (actor) networks utilize PD to update the policy π. By extending PD and PE to the AVCAC, the performance of AGV collision avoidance is effectively improved. In addition, to avoid short-sighted and unnatural motion decisions, and to better enhance the safe awareness of AGV, LMP reward function is introduced in the training phase. Finally, the real action decisions are output by PID controller to enable the AGV to travel smoothly, continuously, and safely during collision-avoidance navigation.

A. AVCAC Configuration

Formally, the POMDP described as a 6-tuple can be solved using the RL framework during collision avoidance navigation. Below, we discuss the RL (i.e., AVCAC) configuration details, including observation space, action space, and reward function.

1) Observations: The information contained in observation space of an AGV includes preset prior data and measurement data from onboard sensors. In such case, AGVs cannot acquire the intents and states of other static and moving obstacles directly, but learns to make predictions and inferences from limited local observations, and make its own near-optimal or optimal collision-free action decision. This local observation way is in accord with the real-world situation compare to prior methods [34], [38] using a fully observable situation, since it is difficult for physical AGVs to accurately perceive the

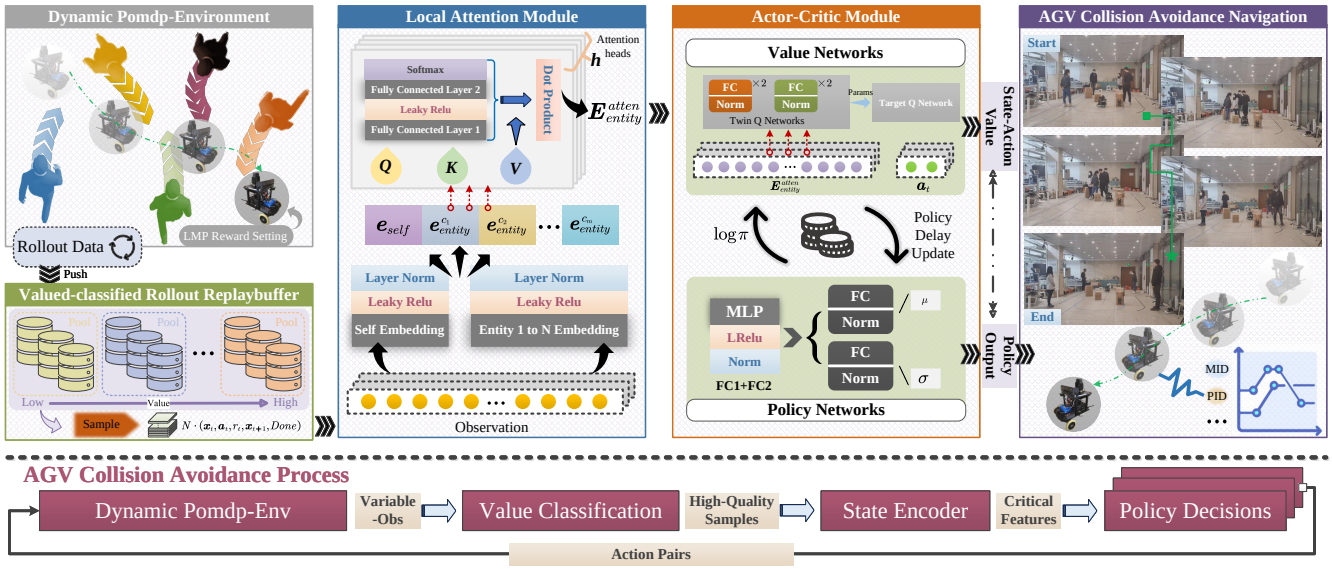


Fig. 2. The overall architecture of the attention-based value classification actor-critic (AVCAC) collision-free navigation method

position and velocity of nearby moving obstacles. The specific observation setting of the AGV at timestep t is as follows:

$$\boldsymbol{x}_t = [\boldsymbol{p}_t, \boldsymbol{v}_t, \boldsymbol{p}_t^g, \beta_t, \boldsymbol{x}_{other}] \tag{4}$$

where $\boldsymbol{p}_t^g$ denotes the global position of target point. $\boldsymbol{p}_t$ and $\boldsymbol{v}_t$ are respectively the global position and the velocity for AGV. β_t is the AGV's heading angle. $\boldsymbol{x}_{other}$ is an imperfect perception vector that integrates the position of (static and dynamic) obstacles observed in FOV of AGV at timestep t.

2) Actions: As for action space, the AGV in our environment utilizes the differential-driving wheels. Thus, a natural choice of action space for the AGV is a translational and rotational velocity, i.e., $\boldsymbol{a}_t = [v_t^l, v_t^w]$. In this work, the action space is continuously represented and is limited in a range of $[v_{\min}, v_{\max}]$. Considering the real robot's kinematic and real-world application, the velocity range is scaled to $[-1, 1]$ during training phase. Moreover, at each time step, we assign goal to the AGV dynamically, which can allow the AGV to generate more flexible decisions.

3) Rewards: The reward design determines whether AGV can be guided effectively, so it plays a crucial role in the AGV to fulfill the task requirements. To make the AGV gain reward continuously, and avoid shortsighted and recklessness behaviors, we split the reward setting of the AGV into a basic configuration and a LMP reward setting. It is noteworthy that we assume that AGV can accurately identify the interaction obstacles is static or dynamic by object detection module. The total reward design is formulated as:

$$r_t = \begin{cases} -1, \ if \ any\left(\boldsymbol{d}_{\mathrm{ASs}}\right) < \boldsymbol{d}_{coll} \ or \ any\left(\boldsymbol{d}_{\mathrm{ADs}}\right) < \boldsymbol{d}_{coll} \\ -F_{scale}\left|d_t^g\right| + r_{time} + r_{LMP}, \ otherwise. \end{cases} \tag{5}$$

where $\boldsymbol{d}_{ASs} = [d_{AS_1}, d_{AS_2}, \cdots, d_{AS_m}]$ is Euclidean distance vector between AGV and observed static obstacles $\{1, ..., m\}$ in its FOV. Similarly, $\boldsymbol{d}_{ADs} = [d_{AD_1}, d_{AD_2}, \cdots, d_{AD_n}]$ is the relative distance vector between AGV and other dynamic obstacles $\{1, ..., n\}$ in its FOV. $\boldsymbol{d}_{coll}$ is the collision distance threshold. $d_t^g = \|\boldsymbol{p}_t^g - \boldsymbol{p}_t\|_2$ denotes the relative L2 distance between AGV and target point at timestep t. $F_{scale} = 1/R_{env}$ is the scaling constant. R_{env} is the radius of the current scene. $r_{time} = -0.01$ given at each timestep is a constant penalty. This item can promote the AGV to reach the goal as quickly as possible.

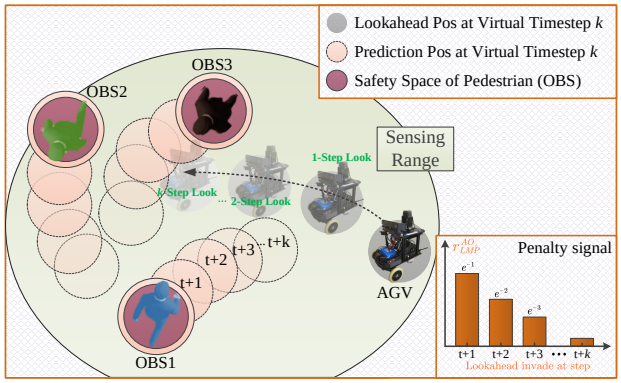


Fig. 3. AGV makes a multi-step trajectory prediction for obstacles (OBS) in its FOV at timestep t, and preforms multi-step uniform trajectory evolution on the basis of the current velocity. At each look-ahead step, AGV virtually invades the safety zone of obstacles or collides with it, while a specific penalty signal is produced. The magnitude of this penalty decays exponentially with the increasing of lookahead step k.

Motivated by [38], we further design a prediction-based lookahead multi-step reward function r_{LMP} to induce AGV to learn more reasonable and longsighted collision-free motion policy in complex dynamic scene. In Fig. 3, the LMP reward configuration between the AGV and j-th dynamic obstacle is as follows:

$$r_{LMP}^{AD_j} = \begin{cases} -e^{-k}, \ if \min\limits_{k=1,\cdots,K}\left[d_{A'D'_j}^{t+k}\right] < d_{comfort}. \\ 0, \qquad otherwise. \end{cases} \tag{6}$$

where t is the current timestep. $d_{comfort} = 0.3$ is the comfort threshold of interaction distance between AGV and dynamic

obstacle. $d_{A'D'_j}^{t+k}$ represents lookahead prediction distance vector between the AGV and j-th dynamic obstacle observed in its FOV. The final LMP reward of the AGV is the minimum of all $d_{A'D'_j}^{t+k}$, where $j = \{1, ..., n\}$ is the index of dynamic obstacles that appear in AGV's FOV.

$$r_{LMP} = \min_{j=\{1,2,\cdots,n\}} r_{LMP}^{AD_j} \tag{7}$$

It is noteworthy that implementing the LMP reward setting in our navigation system relies on two key points. The first one is to predict trajectory evolution of dynamic obstacles in FOV at each timestep in the lookahead virtual time domain (LVTD) by a pre-trained motion prediction module, e.g., MID [39]. Second, for the trajectory evolution of AGV in LVTD, we choose to perform a uniform AGV state update based on current real action pair, which can speed up training process.

Also, the timestep in simulator denotes an individual simulation sub-progress as each discrete step, rather than the frequency at which the real-world sensor data is being sampled. During the simulation phase, the AGV is modeled as a square with a size of 0.25m×0.25m. The objective of navigation task for the AGV is to successfully navigate to the target within a specified timestep number, which in this work is 50 timesteps. The bold data in Table III shows that the average trajectory length calculated by our AVCAC is 15.97m on the simulation scene. Therefore, the average size of each timestep in this work is about 15.97÷50=0.32m. In the perspective, our lookahead multi-step reward function can make the following effect. The exponential penalties are imposed when AGV collides with dynamic obstacle within timestep k=3. It implies that the distance of the AGV and the obstacle is about 0.96m, which is nearly 4 times the size of AGV. This distance includes both immediate and non-immediate collisions. Nevertheless, unlike the existing prediction approach [38] that penalizes the non-immediate collisions at all prediction timesteps, the exponential penalty in our approach approaches to zero when the virtual timestep k is larger than 3, that is to say the AGV is more than 0.96m away from the obstacle. The benefit of weakening or even stopping the penalty for the AGV that is far away from the obstacle lies in achieving an excellent trade-off between navigation safety and efficiency.

B. AVCAC Description

The overall AVCAC algorithm architecture for AGV collision avoidance is shown in Fig. 2. In the following part, we describe the implementation details of the AVCAC, including the VCRR mechanism, local attention module, and the new collision avoidance actor-critic approach.

1) Value-Classified Rollout Replaybuffer: In training phase, we need to push the interaction experience generated by AGV to the replaybuffer. However, the original replay mechanism used in RL typically performs equal probability sampling that does not distinguish the importance (value) of experiences in the replaybuffer [40]. The effect in such case tends to repeatedly learn simple samples and slow down the convergence speed. To address this limitation, a novel VCRR mechanism is devised to partition the experience samples and store them into different sub-replay pools based on their value. This way, the replaybuffer can rollout time-efficient and high-value samples more frequently to prevent any overfitting or bias that may result from repeatedly sampling experiences of a certain type during policy learning. The detailed process is as follows.

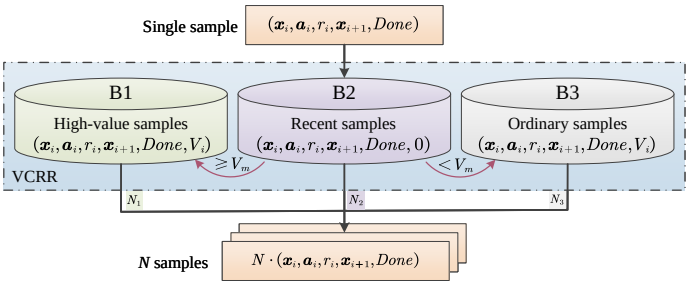


Fig. 4. The structure of value-classified rollout replaybuffer.

First, we design a value measurement standard to classify the experience samples reasonably. The temporal difference (TD) error δ is mostly used for estimating the current state value in RL. The magnitude of TD-error reflects the extent to which the AGV is able to learn from the experiences. There is a great opportunity for improvement towards the expected action, when a larger $|\delta|$ value, since the AGV can timely examine the learning effectiveness of current policy through replaying this type of high-value data. However, the immature policy model at early stage of training has limited environment exploration, which may introduce a large $|\delta|$ at the edge of the state space to change the convergence direction of network. Compared with TD error, the reward r can directly provide feedback on the quality of current action taken. In the short term, the samples with high reward value have higher learning significance in the early stage of training, which can promote the AGV to quickly learn the collision-free navigation policy in the early stage. Thus, this work constructs a new value measurement method by combining the δ and r. It is formulated as:

$$V_j = \alpha \left| r_j \right| + (1 - \alpha) \left| \delta_j \right| \tag{8}$$

where V_j is the value of the j-th sample. $|r_j|$ is the absolute value of the reward for sample j. α is the weight of $|r_j|$. $|\delta_j|$ is the absolute value of TD error of sample j. α is dynamically updated with the training process. The α is first set to 1 when in the early stages of training, which can leverage the reward to guide the AGV to learn collision avoidance policy quickly. With the further training of the model, α gradually decreases to increase the weight of $|\delta_j|$ to better guide the policy update through the TD error.

After value measurement, the experience samples are partitioned and stored in different sub-replay pools. As Fig. 4 shows, the VCRR consists of three sub-replay pools: B1 for storing high-value samples, B2 for storing recent samples, and B3 for storing ordinary samples. All pools use last-in-first-out queue as storage container and have a fixed capacity. When the capacity of the queue reaches its maximum limit, the oldest samples are popped from the head of the queue. The construction steps of the VCRR are given as follows.

Step 1 (Initialization): The average value V_m of all samples in the two sub-replay pools B1 and B3 is set to 0, while the

weight coefficient α of the reward is set to 1. In the process of model training, each sample generated is added to the end of queue B2 in turn. When B2 reaches its capacity limit, the oldest sample $(\boldsymbol{x}_i, \boldsymbol{a}_i, r_i, \boldsymbol{x}_{i+1}, Done)$ is ejected from the head of B2 and the value of the ejected sample is calculated as V_i. If the V_i is higher than V_m, the ejected sample is added to B1, and otherwise stored in B3. Once the storage of all sub-replay pools is completed, the V_m and α are updated, and the above process is repeated until the end of the training.

Step 2 (Stratification Sampling): To ensure both timeliness and diversity of samples in RL, the stratification sampling strategy is devised in this work to randomly extract appropriate samples from sub-replay pools B1, B2 and B3. This can create a balanced and representative minibatch for model training. We record the total number of the minibatch as N, including N_1 samples in B1, N_2 samples in B2, and N_3 samples in B3, i.e.

$$N = N_1 + N_2 + N_3 \tag{9}$$

where N_2 is a fixed-value to ensure that each batch sampling can always contain recent samples. This is important because recent experiences often provide valuable information about the current dynamics of the environment. Including them in the training process can help RL model adapt and update its weights to reflect the most up-to-date knowledge. It is worth noting that the N_1 is set to a large value initially, which means that a large number of high-value samples are selected in the early stages of training. The effect in such case tends to encourage the network to converge in the right direction by emphasizing the most valuable experience. With the progress of policy learning, whereas the performance of the AGV collision avoidance becomes more and more excellent, the occurrence frequency of high-value samples tends to increase. To avoid overfitting with respect to high-value samples, we properly reduce N_1 and increase N_3 as training progresses. This adjustment can make the minibatch more balanced to boost the model generalization.

2) Local Attention Module Design: The local observation within limited FOV of AGV involves various information that is clearly of different importance to it. Hence, the local attention module is an essential component of AVCAC, which learns weighted local encodings to help AGVs focus on the key features of local observations. As shown in Fig. 2, the multi-head attention mechanism is introduced into this module to compute independent attention weights for same input sequences. This way, the AGV can focus on various aspect of observation information. Through the encoding module, the critical information of the environment can be effectively extracted, while keeping the encoding dimension for the complex dynamic scene constant.

The encoding process in this work can be divided into two stages: the pre-encoding stage and the attention encoding stage. To focus on distinguishing and handling different categories of obstacles (entities), we set up individual pre-encoding networks for each entity class in AVCAC. Each pre-encoding network consists of an Embedding layer (fully connected network), a leaky ReLU, and a LayerNorm Layer. The specific process of the pre-encoding stage is as follows.

First, the pre-encoding of AGV self is achieved by using the operation in Eq. (10), that is:

$$\boldsymbol{e}_{self} = f_{self}\left(\boldsymbol{x}_{self}\right) \tag{10}$$

where f_{self} is self pre-encoding network, and $\boldsymbol{x}_{self}$ denotes the information vector of AGV itself (including self-global velocity and position).

Then, the entity information in the local observation $\boldsymbol{x}$ of AGV is pre-encoded as:

$$\begin{aligned} \boldsymbol{e}_{entity}^{c_1} &= f_{entity}^{c_1}\left(\boldsymbol{o}_{entity}^{c_1}\right) \\ \boldsymbol{e}_{entity}^{c_2} &= f_{entity}^{c_2}\left(\boldsymbol{o}_{entity}^{c_2}\right) \\ &\vdots \\ \boldsymbol{e}_{entity}^{c_m} &= f_{entity}^{c_m}\left(\boldsymbol{o}_{entity}^{c_m}\right) \end{aligned} \tag{11}$$

where $c_m\ (m = 1, 2, ..., M)$ denotes the various types of entity class. In our collision-free navigation scene, the different types of entity classes contain target landmarks, static and dynamic obstacles, and other moving robots. For each entity class, the number observed by AGV is different. $\boldsymbol{o}_{entity}^{c_m} = \left[o_{entity_1}^{c_m}, o_{entity_2}^{c_m}, ..., o_{entity_k}^{c_m}\right],\ k \leqslant N_m$ is the number of entity class c_m in the FOV for the AGV. For ensuring constant-sized inputs, N_m is set to 15 herein, which represents the default number of entity class c_m that the AGV can observe. If the number of entity class c_m observed by AGV is less than N_m, the space-occupying zero fill vector is used instead. $o_{entity_k}^{c_m}$ represents information vector for the k-th entity in class c_m. $f_{entity}^{c_1}, f_{entity}^{c_2}, ..., f_{entity}^{c_m}$ are the pre-encoding networks of other entity classes.

Then, we utilize the attention encoding module to acquire the local encoding $\boldsymbol{E}_{entity}^{atten}$ for the AGV. The attention encoding module contains three attention matrices: value matrix $\boldsymbol{W}_v^{eneity}$, key matrix $\boldsymbol{W}_k^{eneity}$, and query matrix $\boldsymbol{W}_q^{self}$. Similar to the pre-encoding networks, each type of entity is equipped with an individual key matrix and value matrix. However, because we only need to measure the contribution of these entities to AGV decision-making, AGV-itself is only equipped with the query matrix, in order to reduce the computation. The pre-encoding is mapped through the attention matrix, as below:

$$\begin{aligned} \boldsymbol{Q} &= \boldsymbol{W}_q^{self}\boldsymbol{e}_{self} \\ \boldsymbol{K}_{entity}^{M} &= \boldsymbol{W}_k^{entity}\boldsymbol{E}_{entity}^{M} \\ \boldsymbol{V}_{entity}^{M} &= \boldsymbol{W}_v^{entity}\boldsymbol{E}_{entity}^{M} \end{aligned} \tag{12}$$

where $\boldsymbol{E}_{entity}^{M} = \left[\boldsymbol{e}_{entity}^{c_1}, \boldsymbol{e}_{entity}^{c_2}, ..., \boldsymbol{e}_{entity}^{c_M}\right]$ denotes the pre-encoding set of the various entities for the AGV. Finally, the local encoding $\boldsymbol{E}_{entity}^{atten}$ is obtained by dot product weighting.

$$\boldsymbol{E}_{entity}^{atten} = \boldsymbol{\alpha}_{entity} \cdot \boldsymbol{V}_{entity}^{M} \tag{13}$$

where the attention weight $\boldsymbol{\alpha}_{entity}$ measures the correlation between various types of pre-encoding $\boldsymbol{E}_{entity}^{M}$ and the AGV. Herein, $\boldsymbol{\alpha}_{entity}$ is calculated by dense synthesizer attention mechanism [41], that is:

$$\boldsymbol{\alpha}_{entity} = \operatorname{Soft\,max}\left(\boldsymbol{F}\left(\boldsymbol{Q} \cdot \left[\boldsymbol{K}_{entity}^{M}\right]^{T}\right)\right) \tag{14}$$

where the structure solely utilizes a two-layer fully connected network $\boldsymbol{F}$ and Softmax to calculate the attention weights.

This way does not require interaction between tokens, and helps the model quickly learn important information from the local observations. Thus, through the local attention module, the AGV can better evaluate the critical information of local observation and make more accurate collision-free navigation decisions. Note that a weight sharing trick [23] is introduced in our local attention module to similarly treat each entity in early stages for endowing our policy with permutation invariance.

3) New Collision-Avoidance Actor-Critic Method: The existing actor-critic method for collision-free navigation faces many problems, e.g., unstable update performance, low generalization ability, and weak exploration efficiency. To address these issues, the PD and PE are extended to the navigation system to further enhance the AVCAC model.

With the integration of PE, the AGV gains an extra reward proportional to PE at each timestep. This changes the optimization objective to:

$$arg\max_{\pi}\mathbb{E}_{(\boldsymbol{x}_t,\boldsymbol{a}_t)\sim\pi}\left[\sum_{t=0}^{T}\lambda^t\left(r\left(\boldsymbol{x}_t,\boldsymbol{a}_t\right)+\eta H\left(\pi\left(\boldsymbol{a}_t|\boldsymbol{x}_t\right)\right)\right)\right] \tag{15}$$

where η is a temperature coefficient that is used to make a trade-off between maximizing policy entropy and maximizing reward. The policy entropy, $H\left(\pi\left(\boldsymbol{a}_t|\boldsymbol{x}_t\right)\right)=-\log\pi\left(\boldsymbol{a}_t|\boldsymbol{x}_t\right)$, can measure the policy randomness and make the algorithm more generalizable and robust. The optimization term, $arg\max_\pi \mathbb{E}_{(\boldsymbol{x}_t,\boldsymbol{a}_t)\sim\pi}\sum_{t=0}^{T}\lambda^t r\left(\boldsymbol{x}_t,\boldsymbol{a}_t\right)$, is used to maximize the reward. Through maximizing PE, the approach possesses stronger robustness and generalization, and enhances the exploration ability of navigation policy.

Then, the optimization goal is achieved by actor–critic training paradigm shown in Fig. 2. The value (critic) network contains the target and twin Q networks, which receives the local encoding $\boldsymbol{E}_{entity}^{atten}$ and the action $\boldsymbol{a}$ to output two action-value functions for the AGV. It is formulated as:

$$\begin{aligned}Q_{\psi_1}&=f_1\left(\boldsymbol{E}_{entity}^{atten},\boldsymbol{a}\right)\\ Q_{\psi_2}&=f_2\left(\boldsymbol{E}_{entity}^{atten},\boldsymbol{a}\right)\end{aligned} \tag{16}$$

where ψ_1 and ψ_2 are the parameters of the twin Q network. f_1 and f_2 are two-layer fully connected networks. After obtaining the twin Q values, the value network is updated by joint regression loss function. That is:

$$L_Q\left(\psi\right)=\mathbb{E}_{(\boldsymbol{x}_t,\boldsymbol{a}_t,r_t,\boldsymbol{x}_{t+1})\sim N}\left[\sum_{j=\{1,2\}}\left(Q_{\psi_j}-y_t\right)^2\right] \tag{17}$$

where

$$\begin{aligned}y_t=&\mathbb{E}_{\boldsymbol{a}_{t+1}\sim\pi_\theta(\boldsymbol{x}_{t+1})}\left[r_t+\lambda\left(-\eta\log\left(\pi_\theta\left(\boldsymbol{a}_{t+1}|\boldsymbol{x}_{t+1}\right)\right)\right.\right.\\&\left.\left.+\min\left(Q_{\bar{\psi}_1},Q_{\bar{\psi}_2}\right)\right)\right]\end{aligned} \tag{18}$$

where N denotes experience data sampled from replaybuffer. Note that we introduce two separated Q heads Q_{ψ_1}, Q_{ψ_2} and corresponding target Q heads $Q_{\bar{\psi}_1}$, $Q_{\bar{\psi}_1}$, to avoid the overestimation problem of the value learning. π_θ is the policy network with shared parameter θ. $\bar{\psi}_1$ and $\bar{\psi}_2$ are the parameters of target Q network. y_t is TD target value at timestep t. The $\sum_{j=\{1,2\}}\left(Q_{\psi_j}-y_t\right)^2$ is the TD error δ_t.

TABLE I
PARAMETERS SETTINGS OF OUR AVCAC NAVIGATION MODEL

Parameters	Value
K_p, K_i, K_d	0.2, 0.1, 0.02
K (LMP)	5
Num episode	5e4
Num start train	1024
Episode length	100
Learning rate	1e-3
Buffer/Batch size	2e5/1024
λ, η	0.95, 0.01
Policy delay	2
Update frequency	4
Multi head h	4
FOV	2π, 6m

Through gradient ascent, the policy network is updated. The gradient formulation is shown below.

$$\begin{aligned}\nabla_\theta J\left(\pi_\theta\right)=&\mathbb{E}_{(\boldsymbol{x}_t,\boldsymbol{a}_t)\sim N,\boldsymbol{a}_t\sim\pi_\theta}\left[\nabla_\theta\log\left(\pi_\theta\left(\boldsymbol{a}_t|\boldsymbol{x}_t\right)\right)\right.\\&\left.\left(-\eta\log\left(\pi_\theta\left(\boldsymbol{a}_t|\boldsymbol{x}_t\right)\right)+\min\left(Q_{\psi_1},Q_{\psi_2}\right)\right)\right]\end{aligned} \tag{19}$$

Also, the stochastic action resampling trick is exploited here to ensure a smooth back-propagation process:

$$a_t\sim\pi_\theta=\tanh\left(\mu_\theta+\sigma_\theta\odot\varepsilon\right),\varepsilon\sim\mathcal{N}\left(0,I\right) \tag{20}$$

where μ_θ and σ_θ are the outputs of the policy network. ε is the timestep-varying random noise. The tanh is nonlinear activation function that limits the actions in a range $[-1,1]$. This way can enable our model to directly update the policy network in light of value network. Besides, the policy delay trick is exploited in actor-critic training paradigm to ensure the steady updating of the policy network.

IV. EXPERIMENTS

To verify the effectiveness of our proposed AVCAC navigation model in avoiding collision under complex dynamic workspaces, we implement our method with ROS Kinetic and Gazebo9.0, while building the policy model based on PyTorch. It is trained on a PC with Intel Core i7-9800X CPU and GeForce RTX 2080Ti GPU. The training process and experiment results of our model are discussed in detail below.

A. Model and Training Details

1) Model Details: The local attention module consists of pre-encoding and attention encoding modules. The number of pre-encoding networks in pre-encoding module is same as the number of entity classes. All pre-encoding networks contain a fully connected layer with the size of [2,128], a LayerNorm layer, and a nonlinear activation function (leaky relu). The attention encoding module receives pre-encoding cues that include three attention matrices ($\boldsymbol{W}_k$, $\boldsymbol{W}_q$, and $\boldsymbol{W}_v$) with the unit dimension [128,128]. Noted that only the AGV self has query matrix, and the number of $\boldsymbol{W}_k$ and $\boldsymbol{W}_v$ is equal to the observed entity types. The actor-critic module consists of value network and policy network. The value network receives the local encoding from the local attention module and the actions

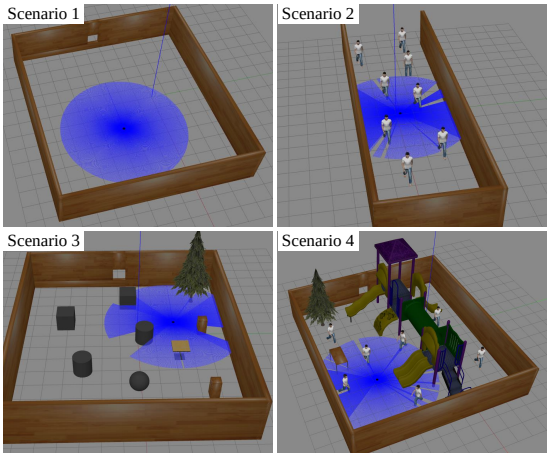


Fig. 5. Scenarios used to train the collision avoidance policy.

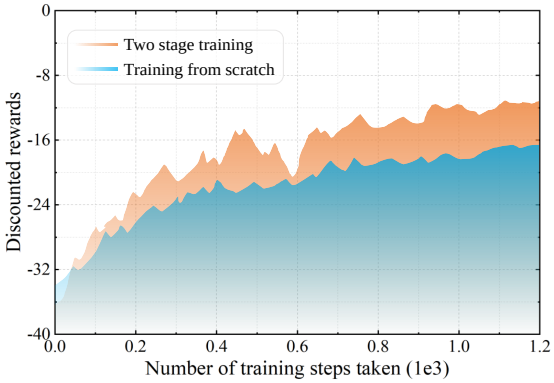


Fig. 6. Discounted reward curves for each step during the training process.

of the AGV. It contains twin Q network, in which each Q head is composed of two hidden layers with dimensions [130,128] and [128,1], respectively. The structure of target Q network is similar to twin Q network. The policy network takes local encoding as input to make the policy update more efficient. It consists of three hidden FC layers with the dimension of [128, 128], [128, 128], and [128, 2], respectively. Table I presents the specific parameter settings of AVCAC.

2) Training Details: To learn a robust collision avoidance policy, we expose our AGV to diverse workspaces by creating different complex scenes with various obstacles. These simulation scenarios are built using the Gazebo mobile robot simulator [42] to accomplish policy training, as shown in Fig. 5. AGV is modeled as a square with a size of 0.25m×0.25m. For scenario 1 without any obstacles, we generate random start and goal positions for AGV at beginning of each episode. AGV in scenario 3 is randomly initialized to avoid the static obstacles. As for scenarios 2 and 4, the policy is trained to achieve autonomous navigation for AGV in a variety of pedestrian crowds. Also, we simulate imperfect sensing by adding timestep-varying random noises to the local observation of AGV. In detail, the Gaussian noises are added on the AGV's observation, i.e., $\boldsymbol{x}_t^{\Gamma} = \boldsymbol{x}_t + \Gamma, \Gamma \sim \mathcal{N}\left(0, \sigma^2\right)$, where Γ denotes Gaussian noises that conform to the standard normal distribution, $\boldsymbol{x}_t$ represents the original observation of AGV at timestep t, and $\boldsymbol{x}_t^{\Gamma}$ is the actual noisy observation received by onboard sensors. Note that the noise parameter σ is randomly sampled from the range [0.1,1], which is not fixed at different timesteps. Through the different σ samples, the noisy observation is independently generated.

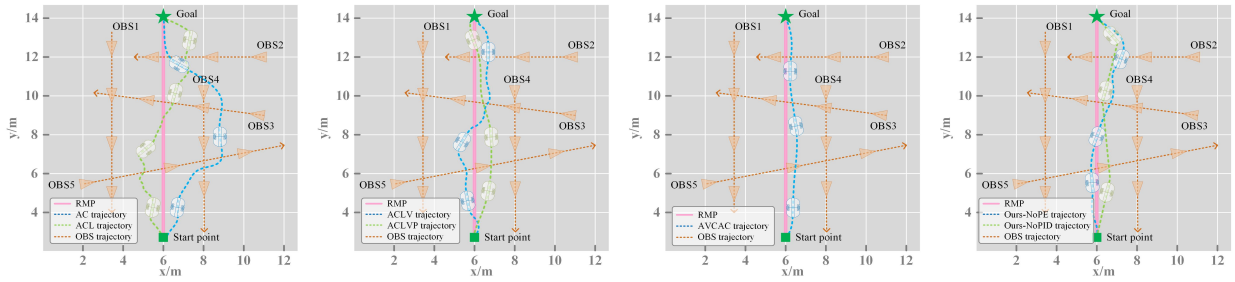
Whereas RL training in various scenarios simultaneously can enable robust collision-free performance, it enables the training procedure more difficult. To tackle such difficulty, a two-stage training process for the AVCAC model is proposed through the inspiration of curriculum learning paradigm [43]. The simulation scenarios in Fig. 5 are used to train the RL-based model in two stages, from simple to complex. In first stage, our goal is to encourage AGV to learn a high-efficiency collision avoidance pattern. For that, we only train the AGV in a simple scenario (i.e., scenarios 1 and 3 in Fig. 5) with static obstacles. The effect in such case tends to quickly find a reasonable collision-free solution. Once the AGV achieves reliable level of collision avoidance, this training process is stopped and the trained policy is saved. Then, we continue to update the policy in the second stage by training in other more-complex scenarios 2 and 4 in Fig. 5, while utilizing a larger collision penalty (-5) to ensure the navigation safety. Clearly shown in Fig. 6, the two-stage training policy can get higher rewards and has faster convergence speed than the policy trained from scratch with the same number of episodes. Note that when switching to second training stage, our reward curve drops slightly, but the curve rises to a higher point very soon. The reason behind this may lie in two aspects: (1) The changes of training scenarios can introduce new variations. This can make the model gained from the first stage to struggle to adapt to the unfamiliar condition initially, leading to a temporary drop in performance. (2) The adjustment of hyperparameters in model during the transition to the second training stage could also have a slight negative impact on its learning process, but it can ultimately lead to better improved performance once the model captures the underlying pattern in the new data.

TABLE II
ABBREVIATIONS OF THE PROPOSED DIFFERENT CONFIGURATION MODELS

Abbreviations	Configurations
AC (Actor-Critic)	RL-only navigation model, i.e., baseline
ACL	Integrate local attention mechanism in AC
ACLV	Fuse value classification module in ACL
ACLVP	Integrate LMP in ACLV
AVCAC (Ours)	Integrate PE and PD in ACLVP
Ours-NoPE	AVCAC without policy entropy
Ours-NoPID	AVCAC without PID operations

B. Evaluation of Our Navigation Model with Different Configurations

Our navigation policy exploits a combination of local attention (LA) and value classification (VC), and is augmented with LMP, policy entropy (PE), and policy delay (PD). In the following part, an intensive experimental analysis of our navigation model with different configurations is built to systematically ablate the components and review the importance



(a) AC and ACL (b) ACLV and ACLVP (c) AVCAC (Ours) (d) Ours-NoPE and Ours-NoPID

Fig. 7. AGV collision avoidance of the proposed different configuration models in dynamic scene.

quantitatively. To simplify the expression, the abbreviations of proposed model with different configurations are given in Table II. Note that AC, ACL, and ACLV all utilize the basic configuration (without LMP reward) as their reward setting, while the other four models use the full reward configuration we describe above (i.e., Eq. (5) that integrates the LMP reward function into the basic configuration). To test above-mentioned models with different configuration, a representative collision avoidance scene shown in Fig. 7 is constructed. In the scene, the multi-moving obstacles at a constant speed are set to encounter the AGV. In detail, the translational speed of obstacle 1 (OBS1), OBS2, OBS3, OBS4, and OBS5 are 0.2m/s, 0.4m/s, 0.6m/s, 0.8m/s and 1.0m/s respectively. The red solid line in the Figure is the reference mission path (RMP) of AGV.

As can be observed, the baseline (AC) struggles to guarantee the optimality of the final track. This happens because they are rarely able to understand collision situation in the dynamic scenes, resulting in useless or even erroneous searches in wrong direction, especially when obstacles occur. The ACL can acquire a relatively short track along RMP. This mainly benefits from the LA that can extract and encode critical environmental features to reduce unnecessary search and improve navigation efficiency. When the VC is integrated into the navigation policy, the length of the track generated by ACLV is shorter than that of ACL. Yet, the desired collision-free steering commands output by ACL and ACLV are unstable, leading to significant fluctuations in the AGV's trajectory. It can be found that the track fluctuation can be further mitigated by ACLVP, compared with the ACLV. This demonstrates the effective function of LMP technique in helping AGV generate more reasonable and farsighted collision avoidance policies. Intuitively, our final AVCAC acquires the best navigation result in the dynamic scene. It can not only complete collision avoidance tasks efficiently, but also produce a continuous, smooth, and dynamic feasible robot trajectory. Besides, the track of Ours-NoPE is unreliable when avoiding moving obstacles, and conducts a certain number of useless searches in the wrong direction to deviate from the RMP. Based on Ours-NoPID, we see that it produces obvious trajectory catastrophe throughout the AGV collision avoidance, which does not satisfy the motion constraints of AGV in practical applications.

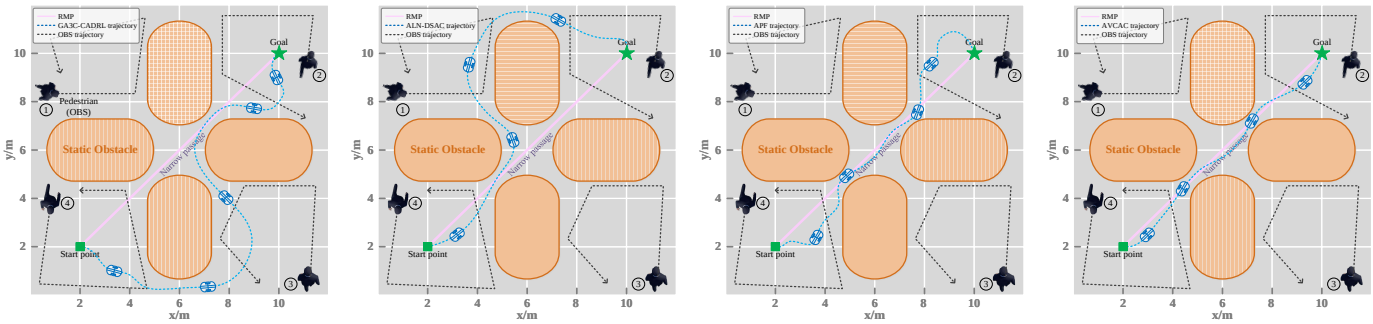
While it's easy to find interesting pros and cons for any given navigation task, it's more useful to draw a conclusion after aggregating extensive randomly-generated cases. Therefore, we design an algorithm to randomly generate 500 navigation episodes for the scene to evaluate the performance of different configuration models quantitatively. In the experiments, five evaluation metrics are adopted, including success rate (SR), time to goal (TG), collision rate (CR), discounted reward (DR), and trajectory length (TL). Note that SR means that rate of case in which AGV successfully reaches its target within 50 timesteps. CR is defined as the percent of cases with a collision. TG and TL are reported based on the success cases. Detailed comparison results are aggregated in Table III.

TABLE III
PERFORMANCE COMPARISON FOR DIFFERENT CONFIGURATION MODELS ON THE DYNAMIC SCENARIO

Methods	SR	CR	Avg TG(s)	Avg DR	Avg TL(m)
AC	0.45	0.58	32.76	-26.15	22.69
ACL	0.69	0.45	23.53	-22.75	19.88
ACLV	0.78	0.31	20.57	-18.91	18.25
ACLVP	0.91	0.11	19.22	-14.83	17.38
Ours	**0.98**	**0.02**	**16.41**	-11.45	**15.97**
Ours-NoPE	0.92	0.08	18.72	-12.27	16.64
Ours-NoPID	0.96	0.04	17.68	**-10.96**	16.22

First, we observe a better navigation performance for the ACL by comparing the results from AC. It verifies the effect of the LA in assisting our approach to focus on the information that is more important for collision-free decision. Subsequently, considering the performance difference among ACL, ACLV, and ACLVP, we see that the idea of each component has a positive effect on the model. When bringing the VC into ACL, the navigation results of ACLV is significantly superior to that of ACL. It achieves 0.78 in SR and 20.57s in average TG, showing 13.04% improvement and 12.58% reduction in average SR and TG respectively. When LMP and VC are both integrated into ACL, the average TG of ACLVP is a gentle reduction with respect to ACLV, while the SR has a significant ascent from 0.78 to 0.91. We consider that the VC enhances the sample utilization that is conductive to limit the sampling space for efficiency while increasing the probability of avoiding collision to some extent. Also, the designed LMP generally motivates AGV to master the skill of generating more reasonable collision avoidance decisions. It is critical for collision-free policy learning in complex dynamic scenarios.

Clearly, our AVCAC combining both PE and PD to jointly optimize navigation policy, gains best performance on the complex dynamic scene in almost all metrics. This shows



(a) GA3C-CADRL (b) ALN-DSAC (c) APF (d) AVCAC (Ours)

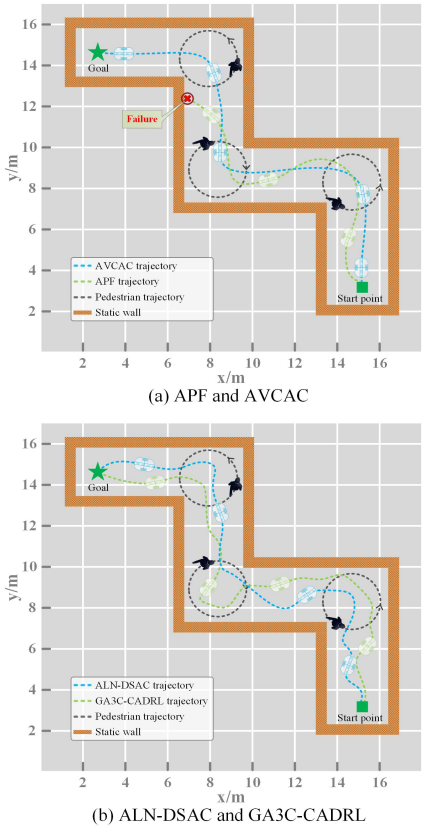
Fig. 8. Collision avoidance navigation of the different methods in the "narrow passage" scenario.

the effectiveness of the combination of each component in improving the performance of the method. Additionally, we see the performance gap of our model (Ours) is smaller than Ours-NoPE, meaning PE enhances policy exploration to make the method more robust. When comparing Ours with Ours-NoPID, it can be found that our model shows 2.04% improvements in SR, while 50% and 7.2% reductions in CR and average TG, respectively. We suggest that the PID facilitates us introduce kinematics constraints to enhance the learned navigation model, enabling AGV to final continuous, dynamic-feasible, and smooth trajectory travel.

C. Collision Avoidance Performance Evaluation

To further verify the superiority of our AVCAC in collision-free navigation, we provide a thorough evaluation by comparing it with four mainstream available methods, including RL-based GA3C-CADRL [24], ALN-DSAC [29], and conventional APF [14] methods. It is noteworthy that we utilize the reward function as defined in Eq. (5) for all RL-based methods in this part to present a fair comparison among them. Besides, all the RL-based policy models are trained on the scenarios shown in Fig. 5 with the proposed two-stage training method. The APF method for AGV is implemented by an abstract artificial potential field with the default parameters provided by the author.

In the inference test stage, two types of challenging test scenarios that do not occur during training phase are built in our experiment: 1) "narrow passage" and 2) "detour". Collision avoidance in narrow passage is a common issue for AGV navigation. In such case, the passable area is generally located in the gap among two obstacles. The navigation evolvement process with narrow passage is detailed in Fig. 8. Note that we randomly set four pedestrians moving around to increase the difficulty of AGV collision avoidance in narrow passage. Each pedestrian is controlled by the plugin "ActorPlugin" within Gazebo in this study. The results show that it is difficult for the GA3C-CADRL and ALN-DSAC methods to sample the area within narrow channel in a limited number of iterations. The limitation makes it hard to plan a reasonable path in narrow passage. We also observe in traditional method that APF can narrow sampling space based on repulsive force generated when approaching obstacles to shorten useless searches. Yet, it still suffers from the impoverishment of trajectories



(a) APF and AVCAC

(b) ALN-DSAC and GA3C-CADRL

Fig. 9. Collision avoidance navigation in "detour" scenario.

swing violently when AGV passes through pedestrian. Also, it can be seen that our AVCAC not only greatly reduces the unnecessary searches but significantly improves the stability and smoothness of trajectories.

Subsequently, the "detouring" scenario is designed to simulate the task of AGV avoiding dynamic obstacles in the course of detour to reach the goal. Herein, the pedestrian is exploited as moving obstacle for experimental evaluation, which walks on a 1.2-meters fixed-radius circle at an angular velocity of 0.3 rad/s. The real-time evolvement results are shown in Fig. 9. We observe that our AVCAC can output a reasonable policy, making AGV avoid movement obstacles smoothly and successfully detour to reach the goal. However, the effect in such case tends to bring about great difficulty

TABLE IV
PERFORMANCE METRICS COMPARISON OF DIFFERENT METHODS ON TWO CHALLENGING SCENARIOS

Models	Narrow passage (12m×12m)				Detour (18m×18m)			
	SR	CR	Avg TG (s)	Avg TL (m)	SR	CR	Avg TG (s)	Avg TL (m)
APF	0.68	0.32	18.62	18.56	0.19	0.82	24.17	26.72
GA3C-CADRL	0.76	0.28	28.11	24.78	0.78	0.35	30.05	34.83
ALN-DSAC	0.93	0.09	22.97	21.45	0.86	0.24	28.64	32.45
AVCAC (Ours)	**0.97**	**0.03**	**16.46**	**17.82**	**0.98**	**0.02**	**21.22**	**25.62**

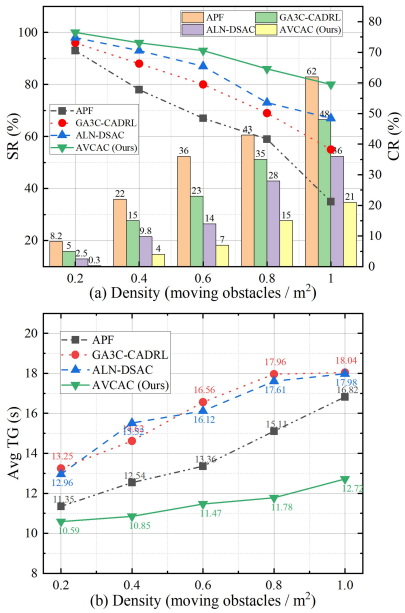


Fig. 10. Navigation performance (a) SR, CR, and (b) average TG comparisons for different methods in scenarios with varied moving obstacle densities.

to other approaches. For instance, GA3C-CADRL and ALN-DSAC can avoid collisions, but generating unnatural trajectories obviously. This means that the desirable decision output is unstable that cannot guarantee the trajectory's optimality. Unexpectedly, APF fails to collision avoidance. The potential reason may be that AGV is much less affected by obstacles' repulsive force than by the attraction of goal in such cases, which makes it difficult to plan a detouring path to dodge obstacles based on the APF algorithm.

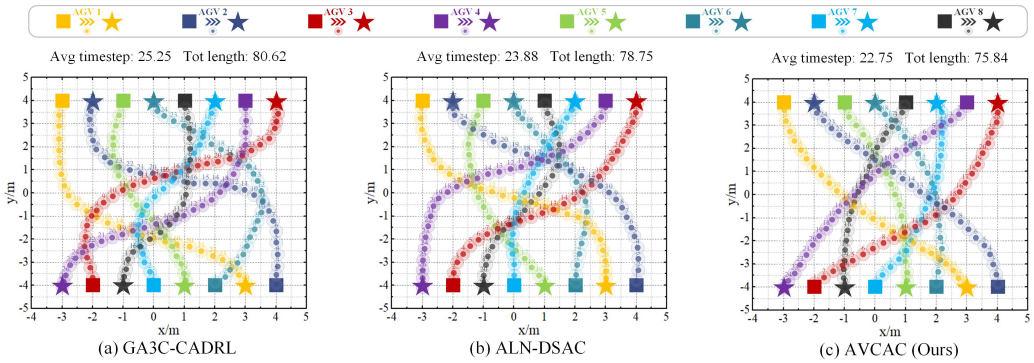
To further compare the quality of collision-free navigation quantitatively, we summarize SR, CR, TG, and TL for different methods in Table IV, through randomly-generated 500 navigation tasks in each complex scenario. As can be seen, the conventional APF has a low SR, but the navigation time within limited successful cases is short. For the RL-based GA3C-CADRL and ALN-DSAC, although a general better performance can be achieved in terms of the SR and CR, they consume more time to generate longer collision-free trajectory. The reason may lie in that due to limited training data, these trained RL methods do not yield sensible performance, thus needing to a long-distance navigation for better collision-free exploration. In summary, our AVCAC achieves best navigation performance on these two scenes for all metrics. It is mainly attributed to three aspects: 1) AVCAC uses the combination of PE and PD as the underlying actor-critic architecture instead of RL-purely, for efficient and robust decision-making, and considers PID operation, which lays a foundation for final executable, smooth, and continuous navigation. 2) AVCAC uses the LMP trick to predict the future events of collision threat. It can not only evaluate the impact of current action on future AGV-environment interaction, also prevent AGV from generating intrusive, shortsighted, and unnatural motion policies. 3) Through the tight cooperation between VCRR and LA, AVCAC system can improve its perception ability and learning efficiency, thus making effective and safe collision avoidance decisions rapidity under complex dynamic scenes.

TABLE V
PERFORMANCE COMPARISON OF DIFFERENT METHODS ON THE COMMON SCENARIO

Methods	SR	CR	Avg TG (s)	Avg TL (m)
APF	0.89	0.12	26.12	29.85
GA3C-CADRL	0.92	0.08	29.21	33.19
ALN-DSAC	0.97	0.03	23.08	27.12
AVCAC (Ours)	**0.98**	**0.02**	**22.85**	**26.53**

Moreover, we design one extra scene with varied moving obstacle densities to further evaluate the performance of different methods. Specifically, this scene involves a 7×7m^2 region, allocating 10, 20, 30, 40, and 50 obstacles that randomly move around, respectively. Thus, the obstacle density for the scene is about 0.2, 0.4, 0.6, 0.8 and 1 moving obstacle per square meter, respectively. Each obstacle density of evaluation involves 600 different navigation tasks. From experimental results in Fig. 10(a), whereas all methods have a decline in SR and a rise in CR when the density changes from 0.2 to 1, our AVCAC by a large margin outperforms the other methods. As shown in Fig. 10(b), AVCAC has least time-consuming. The navigation time of our method only increases gently from 10.59s in 0.2 density to 12.72s in 1 density, while that of others has a dramatic rise, e.g., APF from 11.35s to 16.82s.

Also, we set the pedestrian number to 5 between the initial position and the target point in scenario 2 of Fig. 5, which is a common scene to collision avoidance. By building navigation test in this scene, the performance of our method is evaluated on scenario that is more familiar to scenario used in [14], [24], [29]. When 600 tasks are randomly generated in this scene, the SR, CR, TL and TG metrics are reported in Table V. As can be seen, our AVCAC achieves a similar best collision avoidance performance with ALN-DSAC, while outperforming GA3C-CADRL and APF, in this common scene. This approves again that our AVCAC is an effective solution, which can enable the



(a) GA3C-CADRL (b) ALN-DSAC (c) AVCAC (Ours)

Fig. 11. Visualization of eight-AGV bilateral bidirectional interaction results of different methods.

TABLE VI
ROBOT PARAMETERS

Parameters	W	H	L	$P_{\max}$	R	$v_{\max}$	i
	mm	mm	mm	kg	mm	m/s	/
Value	280	420	350	25	65	1.5	30

AGV to make more informed collision-free action decision for navigating safely and efficiently in common dynamic scenes.

D. Multi-AGV Collision Avoidance Test

In the above experiments, we focus on testing the collision-free navigation of a single AGV. In this section, we design multiple AGVs operating independently to avoid each other's experiments to further examine the navigation ability of our approach. In detail, an 8-AGV bilateral bidirectional interaction scene is developed, in which each one of these AGVs utilizes same method for avoiding other AGVs during interaction. We also adopt GA3C-CADRL and ALN-DSAC as comparison baselines. The ultimate multi-AGV interaction results are shown in Fig. 11, including the comparison of total path length and average timestep of AGVs. As we can see, the interaction results for both GA3C-CADRL and ALN-DSAC intuitively look similar, with ALN-DSAC being slightly more efficient. Besides, each AGV trained by the two methods are relatively conservative, tending to in advance go around a distance. Contrarily, each AGV using our AVCAC can acquire a proper track for interaction to guarantee no collision with other AGVs, while achieves best results in terms of the average timestep and total path length metrics. This suggests the effectiveness and robustness of our AVCAC in multi-AGV collision avoidance.

E. Real-world Navigation Analysis

To further test the generalization capabilities of our navigation model in real world, we use a differential-driving robot prototype to demonstrate our model, as shown in Fig. 12. The sensors are a Rplidar A2 Lidar (used for obstacle detection), a Kinect V2 camera (used for moving object detection [44]), and a NOKOV motion capture system (used for indoor global localization). These sensor kits provide the observations for

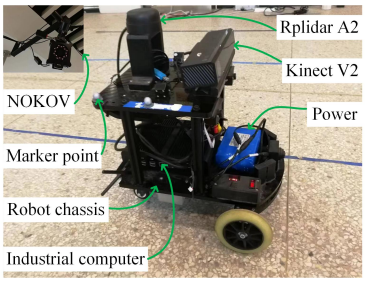
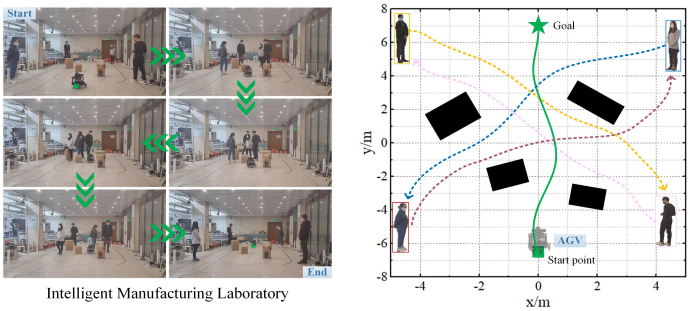


Fig. 12. Robot prototype for collision-free navigation experiments.



(a) (b)

Fig. 13. Collision avoidance navigation of our proposed method in real world (intelligent manufacturing labboratory). (a) the physical robot navigates to its goal in a real complex scene with both moving and static obstacles. (b) the robot's trajectory in the two-dimensional coordinate system, corresponding to (a) on the left. In this Figure (b), the green solid line represents robot's trajectory, the color dashed lines are the pedestrians' paths, and the black boxes refer to the static obstacles.

AGV. Then, AGV executes action commands output from our navigation system that consists of a software framework (ROS Kinetic in Ubuntu16.04) and an industrial computer (Intel Core i7-9800X CPU 8-GB RAM). Although there is no perfect state knowledge, as was available in simulation, the AGV can avoid collisions using only on-board sensing.

The kinematic, geometric, and inertial parameters of the robot prototype at hand are detailed in Table VI. W, H, and L are the width, height, and length of the robot respectively. $P_{\max}$ is the maximum payload carried by the robot. R is the wheel radius. $v_{\max}$ is the maximum traveling speed of the robot, and i is the reduction ratio of the reducer.

The navigation experiments are built in our intelligent man-

TABLE VII
PERFORMANCE COMPARISON OF COLLISION AVOIDANCE NAVIGATION OF OUR METHOD AND ALN-DSAC ON COMPLEX DYNAMIC REAL-WORLD SCENARIOS

Methods	SR	CR	Avg TG (s)	Avg TL (m)
ALN-DSAC	0.81	0.25	23.51	14.69
GA3C-CADRL	0.75	0.36	24.83	15.72
AVCAC (Ours)	**0.95**	**0.05**	**17.28**	**9.97**

ufacturing laboratory (IM-LAB) (approx. $150\mathrm{m}^2$). A typical example is shown in Fig. 13(a). Both static (boxes) and moving (pedestrians) obstacles are placed between robot's starting position and goal about 3m away to interfere with the robot. From trajectory in Fig. 13(b), we observe that the robot can not only smoothly avoid static obstacles, but slow down adaptively to wait for the pedestrians. After the pedestrians pass, the robot starts again towards the goal. Eventually, it successfully and safely navigates to its goal without any collisions.

In addition, the navigation performance is quantitatively measured by randomly setting up another 100 tasks in the laboratory with complex dynamic scene. ALN-DSAC and GA3C-CADRL are employed as two counterparts for performance comparison. Table VII lists the quantitative results, including SR, CR, average TL and TG. A navigation task is regarded as a success only if the robot stops near the goal (e.g., 0.2m) within 30 timesteps. As illustrated in the quantitative results, GA3C-CADRL has worse navigation results than ALN-DSAC, while our AVCAC remains the best in navigation success, safety, and efficiency among all three competing methods. Our AVCAC has a higher than 14% SR and a lower than 20% CR, while shows a 26.5% reduction in average TG, compared to the suboptimal ALN-DSAC. The trial outcomes demonstrate that our AVCAC method can be well deployed to real AGV prototype, presenting an excellent collision-free navigation capability in complex dynamic real-world scenes.

V. CONCLUSION

A novel collision avoidance method, AVCAC, is presented in this paper for efficient and safe AGV navigation in complex dynamic environments. VCRR mechanism is combined with LA module to form a novel actor-critic RL model. It can not only enhance sample efficiency in navigation policy learning to optimize and accelerate the model training, but also improve its perception ability and learning efficiency to make effective and safe collision-free decision rapidity. Then, LMP reward setting is devised to predict the future events of collision threat, which can enable the AGV to realize a good balance between navigation safety and efficiency. Besides, we extend PE and PD to the actor-critic paradigm to encourage policy exploration and make the collision-free navigation model more robust. Finally, a great deal of navigation experiments are built in complex dynamic scenes. The experimental result analysis of different configurations verifies the specific effect of different algorithm elements. By comparing AVCAC with other up-to-date methods, improved safety and generalization, and preserving a higher efficiency than other counterparts, are indicated. Future research will focus on two aspects. The one is how to combine our approach with classical mapping method (e.g., SLAM) to gain satisfactory global collision-free navigation in a dynamic workspace. The other is to develop a temporal-spatial graph-based social encoder to handle the issue of varying-size inputs of obstacle observations.

REFERENCES

[1] H. Wen, Z. Song, S. Liu, Z. Dong, and C. Liu, "A hybrid lidar-based mapping framework for efficient path planning of agvs in a massive indoor environment," *IEEE Trans. Transp. Electr.*, pp. 1–14, 2023.
[2] Z. Zang, J. Song, Y. Lu, X. Zhang, Y. Tan, Z. Ju, H. Dong, Y. Li, and J. Gong, "A unified framework integrating trajectory planning and motion optimization based on spatio-temporal safety corridor for multiple agvs," *IEEE Trans. Intell. Veh.*, pp. 1–12, 2023.
[3] L. Chen, Y. Fu, P. Chen, and J. Mou, "Survey on cooperative collision avoidance research for ships," *IEEE Trans. Transp. Electr.*, vol. 9, no. 2, pp. 3012–3025, 2023.
[4] X. Shang and A. Eskandarian, "Emergency collision avoidance and mitigation using model predictive control and artificial potential function," *IEEE Trans. Intell. Veh.*, vol. 8, no. 5, pp. 3458–3472, 2023.
[5] H. Ryu and Y. Park, "Improved informed rrt* using gridmap skeletonization for mobile robot path planning," *Int. J. Precision Eng. Manuf.*, vol. 20, pp. 2033–2039, 2019.
[6] S. Shen, Y. Mulgaonkar, N. Michael, and V. Kumar, "Multi-sensor fusion for robust autonomous flight in indoor and outdoor environments with a rotorcraft mav," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, 2014, pp. 4974–4981.
[7] T. Fan, X. Cheng, J. Pan, D. Manocha, and R. Yang, "Crowdmove: Autonomous mapless navigation in crowded scenarios," *arXiv:1807.07870*, 2018.
[8] B. Guo, N. Guo, and Z. Cen, "Obstacle avoidance with dynamic avoidance risk region for mobile robots in dynamic environments," *IEEE Robot. Automat. Lett.*, vol. 7, no. 3, pp. 5850–5857, 2022.
[9] T. Fan, P. Long, W. Liu, and J. Pan, "Distributed multi-robot collision avoidance via deep reinforcement learning for navigation in complex scenarios," *Int. J. Robot. Res.*, vol. 39, no. 7, pp. 856–892, 2020.
[10] Y. Tian, X. Zhu, D. Meng, X. Wang, and B. Liang, "An overall configuration planning method of continuum hyper-redundant manipulators based on improved artificial potential field method," *IEEE Robot. Automat. Lett.*, vol. 6, no. 3, pp. 4867–4874, 2021.
[11] Y. Huang, H. Ding, Y. Zhang, H. Wang, D. Cao, N. Xu, and C. Hu, "A motion planning and tracking framework for autonomous vehicles based on artificial potential field elaborated resistance network approach," *IEEE Trans. Ind. Electron.*, vol. 67, no. 2, pp. 1376–1386, 2019.
[12] H. Seo, D. Lee, C. Y. Son, C. J. Tomlin, and H. J. Kim, "Robust trajectory planning for a multirotor against disturbance based on hamilton-jacobi reachability analysis," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2019, pp. 3150–3157.
[13] H. Pham and Q.-C. Pham, "A new approach to time-optimal path parameterization based on reachability analysis," *IEEE Trans. Robot.*, vol. 34, no. 3, pp. 645–659, 2018.
[14] N. Malone, H.-T. Chiang, K. Lesser, M. Oishi, and L. Tapia, "Hybrid dynamic moving obstacle avoidance using a stochastic reachable set-based potential field," *IEEE Trans. Robot.*, vol. 33, no. 5, pp. 1124–1138, 2017.
[15] Y. Chen, H. Peng, and J. Grizzle, "Obstacle avoidance for low-speed autonomous vehicles with barrier function," *IEEE Trans. Control Syst. Technol.*, vol. 26, no. 1, pp. 194–206, 2017.
[16] H. Seo, C. Y. Son, D. Lee, and H. J. Kim, "Trajectory planning with safety guaranty for a multirotor based on the forward and backward reachability analysis," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, 2020, pp. 7142–7148.
[17] H. Xiao, Z. Li, and C. P. Chen, "Formation control of leader–follower mobile robots' systems using model predictive control based on neural-dynamic optimization," *IEEE Trans. Ind. Electron.*, vol. 63, no. 9, pp. 5752–5762, 2016.
[18] S. N. J. Poonganam, B. Gopalakrishnan, V. S. S. B. K. Avula, A. K. Singh, K. M. Krishna, and D. Manocha, "Reactive navigation under non-parametric uncertainty through hilbert space embedding of probabilistic velocity obstacles," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 2690–2697, 2020.
[19] D. H. Lee, S. S. Lee, C. K. Ahn, P. Shi, and C.-C. Lim, "Finite distribution estimation-based dynamic window approach to reliable obstacle avoidance of mobile robot," *IEEE Trans. Ind. Electron.*, vol. 68, no. 10, pp. 9998–10 006, 2020.

[20] H. Yang, X. Xu, and J. Hong, "Automatic parking path planning of tracked vehicle based on improved a* and dwa algorithms," *IEEE Trans. Transp. Electr.*, vol. 9, no. 1, pp. 283–292, 2023.
[21] M. Missura and M. Bennewitz, "Predictive collision avoidance for the dynamic window approach," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, 2019, pp. 8620–8626.
[22] D. Isele, A. Nakhaei, and K. Fujimura, "Safe reinforcement learning on autonomous vehicles," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2018, pp. 1–6.
[23] E. Leurent and J. Mercat, "Social attention for autonomous decision-making in dense traffic," *arXiv:1911.12250*, 2019.
[24] M. Everett, Y. F. Chen, and J. P. How, "Collision avoidance in pedestrian-rich environments with deep reinforcement learning," *IEEE Access*, vol. 9, pp. 10 357–10 377, 2021.
[25] C. Lin, Y. Cheng, X. Wang, J. Yuan, and G. Wang, "Transformer-based dual-channel self-attention for uuv autonomous collision avoidance," *IEEE Trans. Intell. Veh.*, vol. 8, no. 3, pp. 2319–2331, 2023.
[26] Z. He, L. Dong, C. Sun, and J. Wang, "Asynchronous multithreading reinforcement-learning-based path planning and tracking for unmanned underwater vehicle," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 52, no. 5, pp. 2757–2769, 2021.
[27] Q. Wu, K. Xu, J. Wang, M. Xu, X. Gong, and D. Manocha, "Reinforcement learning-based visual navigation with information-theoretic regularization," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 731–738, 2021.
[28] X. Wu, H. Chen, C. Chen, M. Zhong, S. Xie, Y. Guo, and H. Fujita, "The autonomous navigation and obstacle avoidance for usvs with anoa deep reinforcement learning method," *Knowledge-Based Syst.*, vol. 196, p. 105201, 2020.
[29] C. Zhou, B. Huang, and P. Fränti, "Representation learning and reinforcement learning for dynamic complex motion planning system," *IEEE Trans. Neural Netw. Learn. Syst.*, pp. 1–15, 2023.
[30] Z. He, L. Dong, C. Song, and C. Sun, "Multiagent soft actor-critic based hybrid motion planner for mobile robots," *IEEE Trans. Neural Netw. Learn. Syst.*, pp. 1–13, 2023.
[31] C. Song, Z. He, and L. Dong, "A local-and-global attention reinforcement learning algorithm for multiagent cooperative navigation," *IEEE Trans. Neural Netw. Learn. Syst.*, pp. 1–11, 2023.
[32] J. Xia, X. Zhu, Z. Liu, Y. Luo, Z. Wu, and Q. Wu, "Research on collision avoidance algorithm of unmanned surface vehicle based on deep reinforcement learning," *IEEE Sensors J.*, vol. 23, no. 11, pp. 11 262–11 273, 2023.
[33] S. Ross, N. Melik-Barkhudarov, K. S. Shankar, A. Wendel, D. Dey, J. A. Bagnell, and M. Hebert, "Learning monocular reactive uav control in cluttered natural environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2013, pp. 1765–1772.
[34] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan, "Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, 2018, pp. 6252–6259.
[35] J. Godoy, I. Karamouzas, S. J. Guy, and M. L. Gini, "Moving in a crowd: Safe and efficient navigation among heterogeneous agents." in *Proc. Int. Joint Conf. Artific. Intell. (IJCAI)*, 2016, pp. 294–300.
[36] A. H. Qureshi, Y. Miao, A. Simeonov, and M. C. Yip, "Motion planning networks: Bridging the gap between learning-based and classical motion planners," *IEEE Trans. Robot.*, vol. 37, no. 1, pp. 48–66, 2020.
[37] F. A. Oliehoek, C. Amato *et al.*, *A concise introduction to decentralized POMDPs*. Springer, 2016, vol. 1.
[38] S. Liu, P. Chang, Z. Huang, N. Chakraborty, W. Liang, J. Geng, and K. Driggs-Campbell, "Socially aware robot crowd navigation with interaction graphs and human trajectory prediction," *arXiv:2203.01821*, 2022.
[39] T. Gu, G. Chen, J. Li, C. Lin, Y. Rao, J. Zhou, and J. Lu, "Stochastic trajectory prediction via motion indeterminacy diffusion," in *Proc. Comput. Vision Pattern Recognit. (CVPR)*, 2022, pp. 17 113–17 122.
[40] S. Sinha, J. Song, A. Garg, and S. Ermon, "Experience replay with likelihood-free importance weights," in *Learning for Dynamics and Control Conference*, 2022, pp. 110–123.
[41] Y. Tay, D. Bahri, D. Metzler, D.-C. Juan, Z. Zhao, and C. Zheng, "Synthesizer: Rethinking self-attention for transformer models," in *Int. Conf. Mach. Learn. (ICML)*. PMLR, 2021, pp. 10 183–10 192.
[42] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, vol. 3, 2004, pp. 2149–2154.
[43] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2009, pp. 41–48.
[44] C. Sun, X. Wu, J. Sun, C. Sun, M. Xu, and Q. Ge, "Saliency-induced moving object detection for robust rgb-d vision navigation under complex dynamic environments," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 10, pp. 10 716–10 734, 2023.

Chao Sun received the B.S. degree in mechanical engineering from the University of Jinan, Jinan, China, in 2017, and M.S. degree in mechanical and electrical engineering from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2020. Now he is a Ph.D. candidate of the College of Electronics and Information Engineering, Tongji University, Shanghai, China. His current research interests include reinforcement learning, computer vision, and robot navigation.

Xing Wu (Member, IEEE) received the B.Eng. and Ph.D. degrees in Mechanical Engineering from Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China, in 2004 and 2010, respectively. In 2010, he joined the College of Mechanical and Electrical Engineering, NUAA, where he is currently an associate professor. He was a visiting professor at the Centre for Intelligent Machines, McGill University, Canada, from Nov. 2016 to Nov. 2017. His research interests include: intelligent sensing and control; design, navigation and control of mobile robots; and coordinated control and intelligent scheduling of multi-robot systems.

Yuanda Wang received the B.S. degree in automation from Nanjing University of Information Science and Technology, Nanjing, China, in 2014, and the Ph.D. degree in control science and engineering from Southeast University, Nanjing, in 2020. He is currently working as a Post-Doctoral Researcher with the School of Automation, Southeast University. His current research interests include deep reinforcement learning, robotics, and multiagent systems.

Changyin Sun (Senior Member, IEEE) received the B.S. degree in Mathematics from Sichuan University, Chengdu, China, and the M.S. and Ph.D. degrees in electrical engineering from Southeast University, Nanjing, China, in 2001 and 2004, respectively. He is a Professor with School of Automation, Southeast University. His current research interests include intelligent control, flight control, pattern recognition, and optimal theory.

Dr. Sun is an Associate Editor of the IEEE Transactions on Neural Networks and Learning Systems, Neural Processing Letters, and the IEEE/CAA Journal of Automatica Sinica.